I. Use Case Description	
Use Case Name	*Ontology-Guided Agentic Capabilities Orchestration*
Use Case Identifier	*AC-001*
Source	*CSCI 6340 - Ontologies*
Point of Contact	*Gia Oriana Santos (santog@rpi.edu), Barbara Kotlan (kotlab@rpi.edu), Jiaying(Michael) Wang (wangj63@rpi.edu), Jialin (Gary) Fang (fangj6@rpi.edu), Eli (guedee@rpi.edu)*
Creation / Revision Date	*10/30/2026*
Associated Documents	

II. Use Case Summary	
Goal	*To govern which agent skills and tools to invoke, as well as determine the most optimal orchestration plan an LLM should follow given the user's request and constraints*
Requirements	*This use case must accomplish the following tasks* ● *Identify agent capabilities that can fulfill the information requirements or goals expressed in a user request* ● *Construct one or more candidate orchestration plans by sequencing agent capabilities to satisfy the information requirements of a user request.* ● *Determine whether capabilities in a candidate plan have their required inputs, outputs, preconditions, and dependencies satisfied, and identify missing or incompatible dependencies before execution* ● *Select an orchestration plan that satisfies all or most of the constraints specified in the user's preferences* ● *Explain why an orchestration plan is selected by mapping candidate plans to decision criteria* ● *Plan independent capabilities in a given orchestration plan that can run in parallel*
Scope	● ***These are the external servers available to the LLM*** ○ ***PubMed MCP*** ○ ***Supplement Advisor MCP*** ○ ***OpenFDA MCP*** ○ ***iHerb-CLI*** ● ***External servers are exclusively MCP and CLI*** ● ***The LLM cannot use RAG or GraphRAG*** ● ***Specific domain: personalized supplement recommendation*** ● ***Not considering "common knowledge" questions. Only questions related to personalized supplement recommendations*** ● ***We mention the NLPTool as a secondary actor in the system. The development of the NLP Tool is outside the scope of the use case***
Priority	*N/A*
Stakeholders	● *LLM users* ● *External server providers*
Description	*The emergence of the MCP has enabled LLMs to access external tools and data sources through standardized interfaces, allowing AI agents to perform complex retrieval, reasoning, and decision-making tasks. As AI agents become increasingly modular, they must not only determine which external servers and tools to invoke, but also decide when specialized agentic skills should be executed to accomplish subtasks for an overall goal. As the number of available skills and tools continues to grow, LLM agents face increasingly*

II. Use Case Summary	
	difficult orchestration problems, including determining which skills and tools should be executed, dependencies between skills and tools in orchestration, choosing the best valid orchestration plan given the particular user and situation, and explaining why the orchestration plan should be preferred over another. Current LLMs typically make these decisions implicitly through prompting or learned behavior, making the orchestration process difficult to explain, reproduce, validate, and optimize. *This use case addresses the ambiguity of agentic skill and tool orchestration by incorporating an ontology as a layer between the user and agent to explicitly represent the capabilities, inputs, outputs, dependencies, execution requirements, and characteristics of available skills, external servers, and their associated tools. Given a user's request, the ontology guides the LLM agent in selecting the appropriate skills, identifying the relevant external servers and tools, determining the optimal sequence of skill executions and tool invocations, and selecting the orchestration plan that best satisfies the task requirements. The use case's scope focuses on generating evidence-based, personalized supplement recommendations by integrating information from a user's preference settings to act as constraints on orchestration plan selection. This use case also incorporates external servers to support retrieval from scientific literature, safety screening, and an existing product catalog. By grounding orchestration decisions in explicit ontological knowledge, the framework enables transparent, explainable, reusable, and repeatable agent workflows while ensuring that supplement recommendations are supported by user-specified constraints for semantic orchestration plan generation*
Actors / Interfaces	*Primary actors* ● *LLM User* ● *LLM Agent* *Secondary actors (resources)* ● *External Servers* ○ *PubMed MCP* ○ *Supplement Advisor MCP* ○ *OpenFDA MCP* ○ *iHerb-CLI* ● *NLP Tool* ● *Agent Harness (ie. Claude Code, Codex CLI, OpenClaw, etc.)* ● *Reasoner* ● *Knowledge Graph (KG)*
Pre-conditions	● *Ontology is loaded into the system* ● *LLM agent model must be able to use skills and tools (including MCP and CLI).* ● *LLM agent must have the following skills* ○ *Supplement Recommendation* ○ *Safety Screening* ○ *Evidence Verification* ○ *Product Selection* ● *LLM agent is connected to the following external servers* ○ *PubMed MCP* ○ *Supplement Advisor MCP* ○ *OpenFDA MCP* ○ *iHerb-CLI* ○ *LLM agents must also have access to the user's preference settings to facilitate constraint-aware selection*

II. Use Case Summary	
Post-conditions	• *Harness provides ontology-aligned orchestration plan to the LLM* • *Orchestration plan is recorded to the KG for provenance*
Triggers	• *User sends a request to the LLM agent.*
Performance Requirements	
Assumptions	
Open Issues	

III. Usage Scenarios

Provide at least two usage scenarios that flesh out the requirements outlined in the summary, including identification of requirements specific to any envisioned ontology or semantically-driven service or application. Scenarios should be described as narrative, with supporting diagrams as appropriate. In an Agile process, every user story relevant to the use case should be included and elaborated/rolled up into one or more usage scenarios, with a clear mapping from the user story to the scenario it is integrated in or mapped to.

1. Ken is a college student training to be a hybrid athlete in search of supplements while on a budget. Jessica is also looking for supplements, but is diagnosed with osteoarthritis and taking prior medication. Both use the Supplement Recommender agent through their LLMs, but they each have different user preferences in terms of agent orchestration.

   Ken prefers to **minimize token usage and external tool calls** whenever possible. As a college student on a budget, he is looking for **affordable supplements** with **strong customer review**s and **good value.** He wants recommendations that are **supported by scientific evidence but is willing to accept slightly less comprehensive analysis if it reduces cost**. Jessica, on the other hand, **prioritizes recommendations that are supported by high-quality scientific evidence**. She also expects the system to **screen for potential interactions with her prescription medication** before making any recommendations.

2. Dante uses the Supplement Recommender agent and can view the candidate orchestration plans considered by the agent through the system's harness. Although he can see the plans, he is unfamiliar with **what each capability does**, **which servers provide the invoked tools**, **what intermediate information artifacts are produced at each step of the orchestration,** and **why each agent capability was invoked in a particular order**. He wants to understand **why the agent selected one orchestration plan over the other candidate plans**.

3. Olivia wants to find the best heart-health supplement. She explicitly tells the Supplement Recommender agent to find **strong scientific evidence** while also requiring the response to use as **few external tool calls as possible**. The agent identifies that these preferences trade off against each other, because more evidence needs more tool calls, but they do not contradict each other. Regardless, the agent generates candidate plans with different **trade-offs between evidence coverage and execution cost**. The agent **determines which orchestration plan to choose based on Olivia's user preferences** and goes ahead to execute that orchestration plan.

IV. Basic Flow of Events

Narrative: *Often referred to as the primary scenario or course of events, the basic flow defines the process/data/work flow that would be followed if the use case were to follow its main plot from start to end. Error states or alternate states that might occur as a matter of course in fulfilling the use case should be included under Alternate Flow of Events, below. The basic flow should provide any reviewer a quick*

overview of how an implementation is intended to work. A summary paragraph should be included that provides such an overview (which can include lists, conversational analysis that captures stakeholder interview information, etc.), followed by more detail expressed via the table structure.

In cases where the user scenarios are sufficiently different from one another, it may be helpful to describe the flow for each scenario independently, and then merge them together in a composite flow.

Usage Scenario 1 - Basic / Normal Flow of Events: constructing an orchestration plan for execution			
Step	**Actor (Person)**	**Actor (System)**	**Description**
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints
4	-	Reasoner	Identifies candidate agent tools and skills from the information requirements
5	-	Reasoner	Determines candidate orchestration plans based on a combination of required agent tools and skills that satisfy the information requirements
6	-	Reasoner	Evaluates candidate orchestration plans by applying constraints (user constraints and execution constraints) and ranking them on the modeled decision criteria
7	-	Reasoner	Selects orchestration plan
8	-	Harness	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
9	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
10	agent		Generates answer to user's request
11		KG	Records skill and tool invocations from the orchestration as well as decision criteria as provenance

Summary: The **User** submits a request, after which the **NLP Tool** parses the request to extract goals, constraints, and preferences. The **Harness** retrieves the user's preference settings to identify orchestration constraints. The **Reasoner** identifies candidate tools and skills to satisfy the information requirements, as well as candidate orchestration plans that collectively satisfy those requirements. The **Reasoner** then evaluates and ranks the candidate plans according to user and planning constraints and relevant decision criteria before selecting an orchestration plan. The **Harness** derives dependencies between orchestration steps from their inputs and outputs to form a dependency graph, and the **Agent** executes the selected plan by invoking the required tools and skills in the appropriate order. The **Agent** then generates an answer to the user's request, while the **KG** records the tool and skill invocations and associated decision criteria as provenance for the resulting orchestration.

Usage Scenario 2 - Basic / Normal Flow of Events: explaining why an orchestration plan was selected for execution			
Step	**Actor (Person)**	**Actor (System)**	**Description**
1	user		Submits a request
2	-	Reasoner	Generates candidate orchestration plans
3	-	Reasoner	Selects orchestration plan based on DecisionCriteria
4		KG	Records candidate orchestration plans and selected plan with corresponding DecisionCriteria as provenance
5	Agent	-	Executes orchestration plan by invoking agent tools and

			skills according to the plan
6	user		Requests an explanation of the selected orchestration plan through the system's Harness
7		Reasoner	Identifies DecisionCriteria of corresponding selected orchestration plan
8	-	Reasoner	Maps each ToolInvocation and SkillInvocation to the Capability it provides and to the InformationRequirements and Constraints that Capability satisfies
9	-	Harness	Displays the DecisionCriteria and mapped InformationRequirements & Constraints to each agent capability in the orchestration plan
10	User	-	Views the explanation of the orchestration plan via the system's Harness

Summary: The **User** submits a request to the **Supplement Recommender Agent**. The **Reasoner** generates candidate orchestration plans and selects an orchestration plan based on applicable Decision Criteria, while the **Knowledge Graph** records the candidate plans, selected plan, and corresponding Decision Criteria as provenance. The **Agent** then executes the selected orchestration plan by invoking the required tools and skills according to the plan. Following execution, the **Reasoner** identifies the Decision Criteria associated with the selected plan and maps each Tool Invocation and Skill Invocation to the Capabilities they provide, as well as the Information Requirements and Constraints satisfied by those capabilities. The **Harness** then displays the Decision Criteria and the mapped Information Requirements and Constraints associated with each agent capability in the orchestration plan, allowing the **User** to view an explanation of why the selected orchestration plan was chosen.

Usage Scenario 3 - Basic / Normal Flow of Events: constraint-based selection based on non-conflicting Decision Criteria			
Step	**Actor (Person)**	**Actor (System)**	**Description**
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints (ie. high evidence coverage, low external tool calls)
4	-	Reasoner	Identifies candidate orchestration plans whose Capabilities satisfy the InformationRequirements
5	-	Reasoner	Selects "Evidence Coverage" and "External Tool Calls" as DecisionCriteria to select orchestration plan
6	-	Harness	Calculates evidence coverage and external tool call scores for each candidate plan
7		KG	Records the evidence coverage and external tool call scores of each candidate plan
8	-	Reasoner	Ranks each candidate plan by evidence coverage and external tool call scores
9		Reasoner	Selects orchestration plan based on highest DecisionCriteria score
10	-	Harness	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
11	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
12	agent	-	Generates answer to user's request
13		KG	Records candidate orchestration plans and selected plan with corresponding DecisionCriteria as provenance

Summary: The **User** submits a request to the **Supplement Recommender Agent**. The **NLP Tool** parses

the request to extract the user's goals, constraints, and information requirements, while the **Harness** retrieves the user's preference settings to identify relevant orchestration constraints, such as *high evidence coverage* and *minimizing external tool calls*. The **Reasoner** identifies candidate orchestration plans whose Capabilities satisfy the extracted Information Requirements and selects *Evidence Coverage* and *External Tool Calls* as the Decision Criteria for evaluating the plans. The **Harness** calculates the evidence coverage and external tool call scores for each candidate plan, and the **Knowledge Graph** records these scores as provenance. The **Reasoner** first drops any candidate plan that breaks a hard constraint, such as required safety screening. It then scores the remaining plans on each Decision Criterion and combines the scores using the priority the user gave each constraint; if the user gives no priorities, the criteria count equally, as in CQ 6. Olivia's evidence and tool-call constraints pull in different directions but can both be met to some degree, so the Reasoner trades them off in this weighted ranking instead of reporting a conflict. The **Harness** then derives the dependencies among the orchestration steps from their inputs and outputs to construct a dependency graph. The **Agent** executes the selected orchestration plan by invoking the required tools and skills according to the derived dependencies, generates an answer to the user's request, and the **Knowledge Graph** records the candidate plans, selected plan, Decision Criteria, and associated evaluation information as provenance.

V. Alternate Flow of Events

Narrative: *The alternate flow defines the process/data/work flow that would be followed if the use case enters an error or alternate state from the basic flow defined, above. A summary paragraph should be included that provides an overview of each alternate flow, followed by more detail expressed via the table structure.*

Usage Scenario 1 - Alternate Flow of Events: current agentic capabilities cannot satisfy the user's request			
Step	**Actor (Person)**	**Actor (System)**	**Description**
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints
4		Reasoner	Determines there are no agentic capabilities that satisfy one or more information requirements
5		Reasoner	Identifies unsatisfied information requirements
6	-	Harness	Returns unsatisfied information requirements
7	agent	-	Informs the users that the available agent capabilities cannot satisfy the user's request, highlighting the unsatisfied information requirements needed to answer the query

Summary: The **User** submits a request. The **NLP Tool** parses the request to extract goals, constraints, and information requirements. The **Harness** retrieves the user's preference settings to identify relevant orchestration constraints. The **Reasoner** determines that the available agentic capabilities cannot satisfy one or more of those requirements. It identifies the unsatisfied information requirements, which the **Harness** returns to the **Agent**. The **Agent** then informs the user that the currently available tools and skills cannot fully satisfy the request, explicitly highlighting the information requirements that cannot be fulfilled and are therefore needed to answer the query.

Usage Scenario 2 - Alternate Flow of Events: current agentic capabilities cannot satisfy the user's request (requires explanation)			
Step	**Actor**	**Actor**	**Description**

	(Person)	(System)	
1	user		Submits a request
2	agent	-	Informs the users that the available agent capabilities cannot satisfy the user's request, highlighting the unsatisfied information requirements needed to answer the query
3	-	Reasoner	Retrieves the InformationRequirements the system was trying to satisfy in the previous query
4		Reasoner	Retrieve all the Tools and Skills available to the agent
5		Reasoner	Map those Tools and Skills to their Capabilities and the InformationRequirements they satisfy
6		Reasoner	Identifies unsatisfied InformationRequirements
7	-	Harness	Display the Tools and Skills with the InformationRequirements they satisfy, as well as the unsatisfied InformationRequirements
8	User	-	Views the explanation as to why the agent failed to execute the task via the system's Harness

Summary: When the available agentic capabilities cannot satisfy the **User's** request, the **Agent** informs the user that the request cannot be fulfilled and identifies the Information Requirements that remain unsatisfied. The **Reasoner** retrieves the Information Requirements associated with the request, as well as all Tools and Skills currently available to the agent. It then maps the available Tools and Skills to their corresponding Capabilities and determines which Information Requirements those Capabilities can satisfy, allowing the system to identify the Information Requirements that remain unsatisfied. The **Harness** displays the available Tools and Skills, the Information Requirements they can satisfy, and the unsatisfied Information Requirements. The **User** can then view this explanation through the system **Harness** to understand why the **Agent** was unable to execute the task.

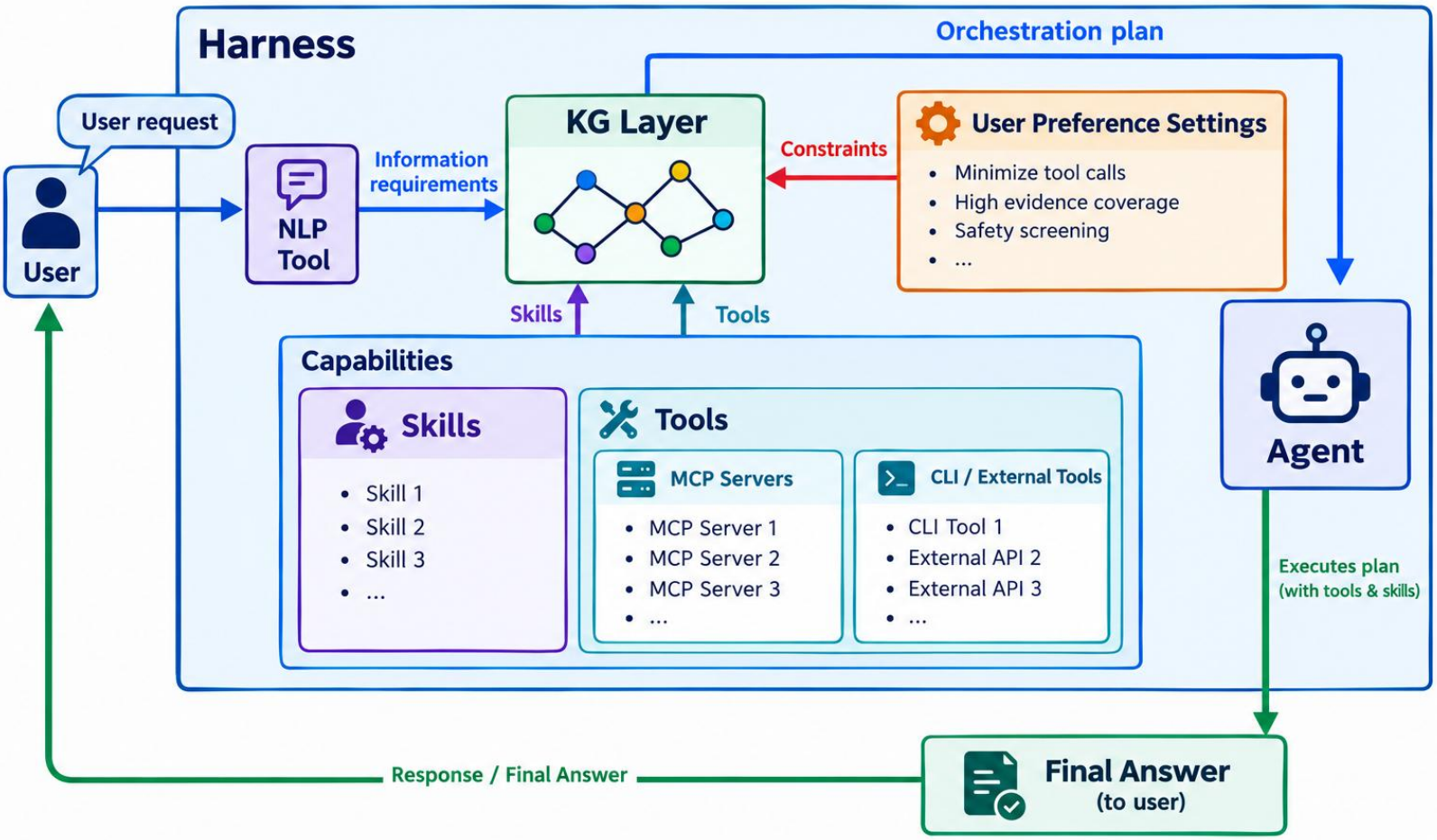
Usage Scenario 3 - Alternate Flow of Events: identifies directly conflicting constraints (ex. Maximize tool use and minimize tool use)			
Step	**Actor (Person)**	**Actor (System)**	**Description**
1	user		Submits a request
2		NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints (ie. maximize tool use, minimize tool use)
4	-	Reasoner	Identifies that the constraints directly conflict with each other
5	-	Harness	Returns conflicting constraints
6	agent	-	Informs the user that it cannot execute the request because the constraints conflict, names the conflicting constraints, and asks the user to revise them

Summary: The **User** submits a request, and the **NLP Tool** parses the request to extract the user's goals, constraints, and Information Requirements. The **Harness** retrieves the user's preference settings to identify the orchestration constraints associated with the request, while the **Reasoner** evaluates these constraints and identifies a direct conflict between them, such as *simultaneously maximizing and minimizing tool use*. The **Harness** returns the conflicting constraints, and the **Agent** informs the user that it cannot execute the task because the specified constraints are incompatible. The **User** is then prompted to resolve or modify the conflicting constraints before the request can be executed. Only constraints that cannot both hold, such as maximizing and minimizing tool use, stop planning; constraints that merely trade off are ranked as in the basic flow.

VI. Use Case and Activity Diagram(s)

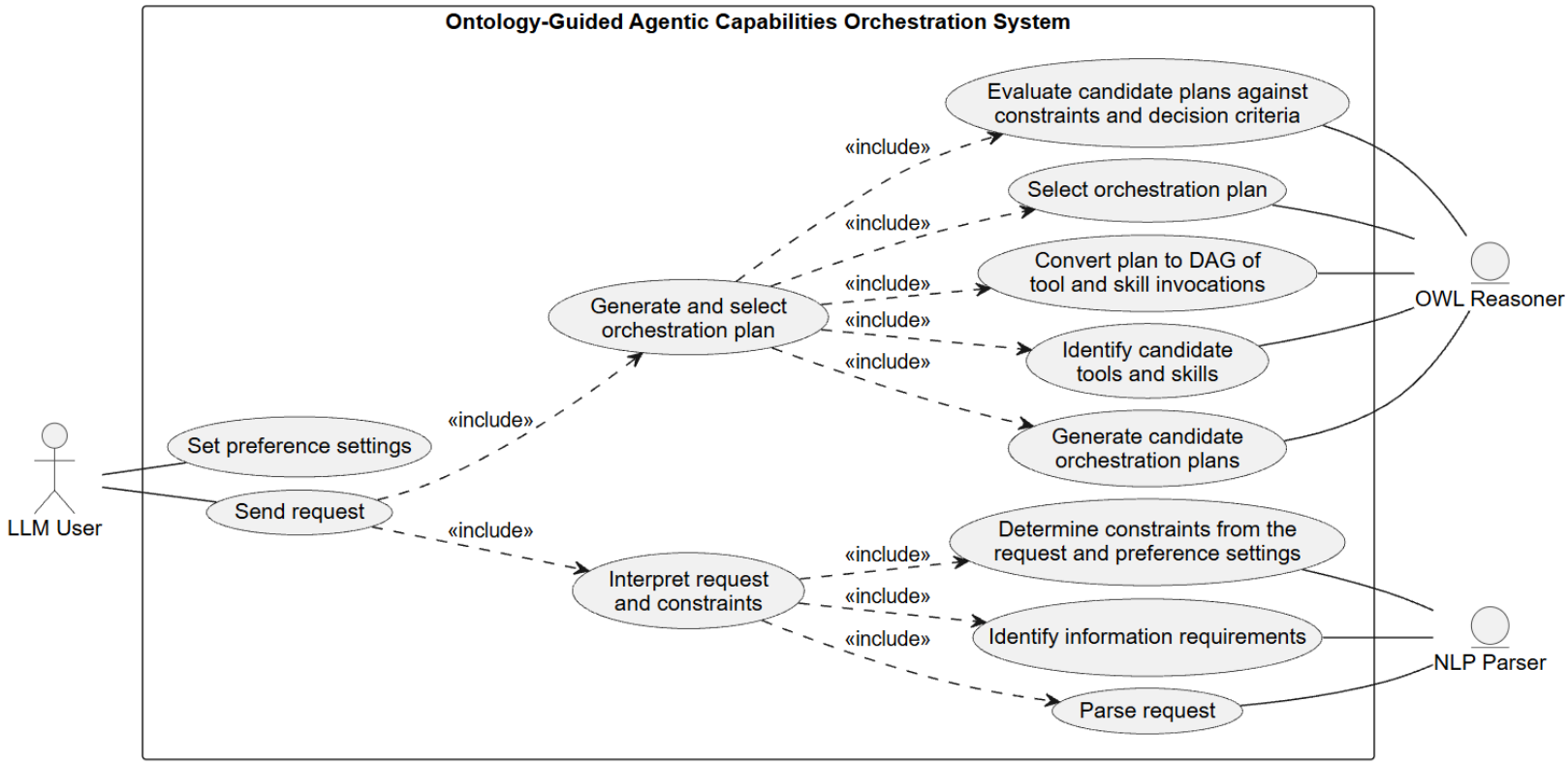
Provide the primary use case diagram, including actors, and a high-level activity diagram to show the flow of primary events that include/surround the use case. Subordinate diagrams that map the flow for each usage scenario should be included as appropriate

Structural Diagram of System Architecture



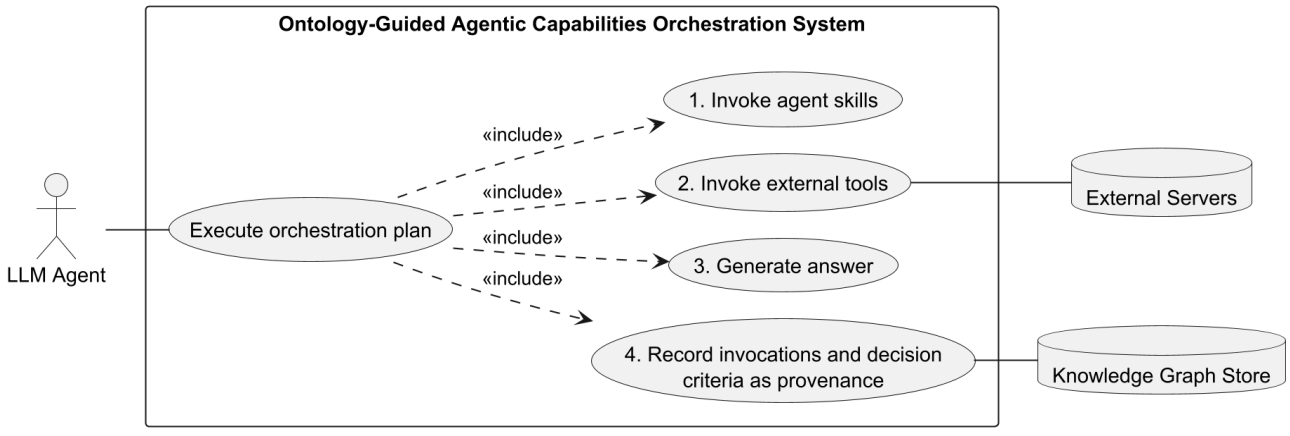
Use case diagram: LLM User (development of NLP Parser is out-of-scope for this project)

Figure 2. Primary use case diagram - AC-001 Ontology-Guided Agentic Capabilities Orchestration: LLM User and Agent Developer Side



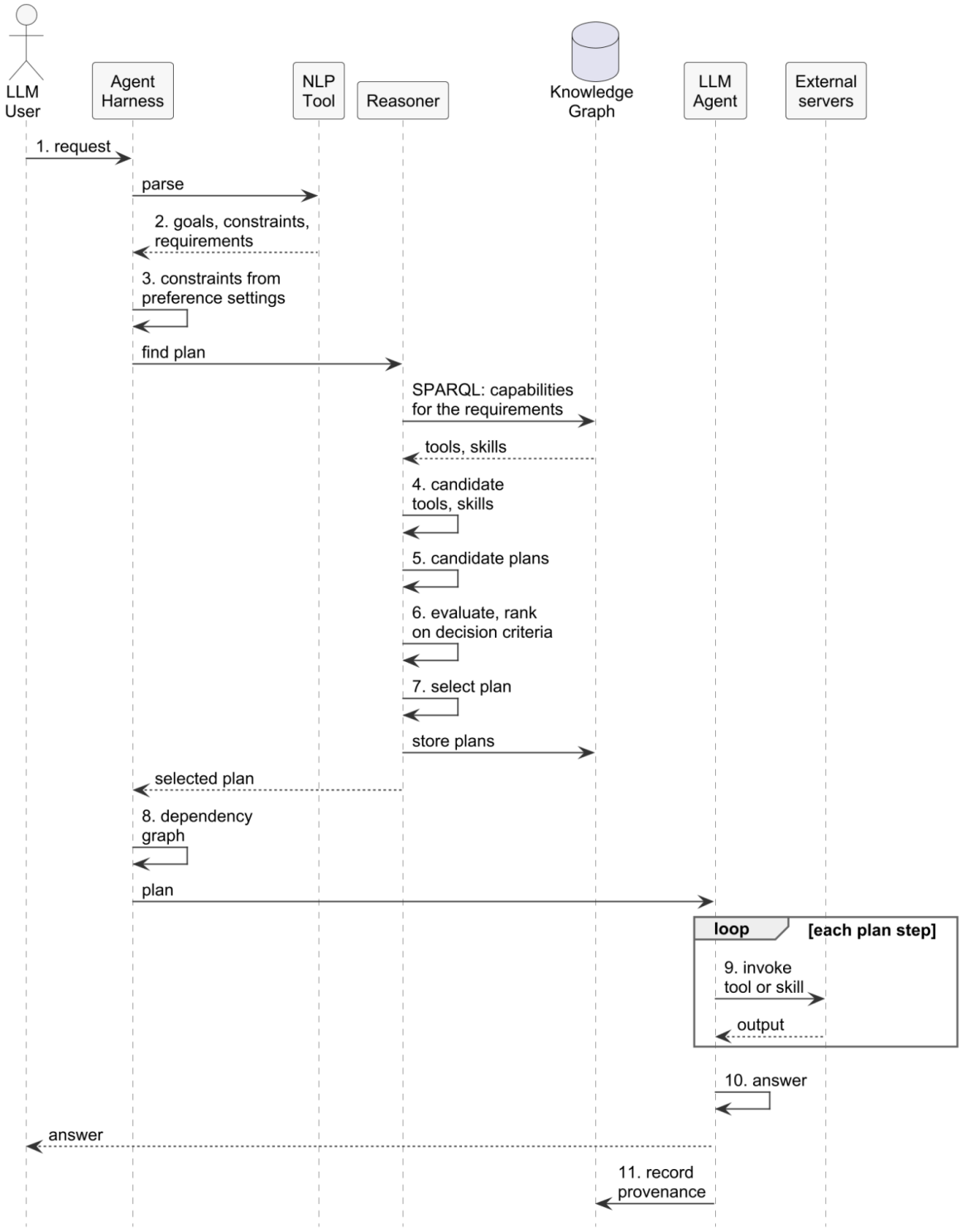
Use case diagram: Agent Side

Figure 2a. Primary use case diagram - AC-001 Ontology-Guided Agentic Capabilities Orchestration: Agent Side



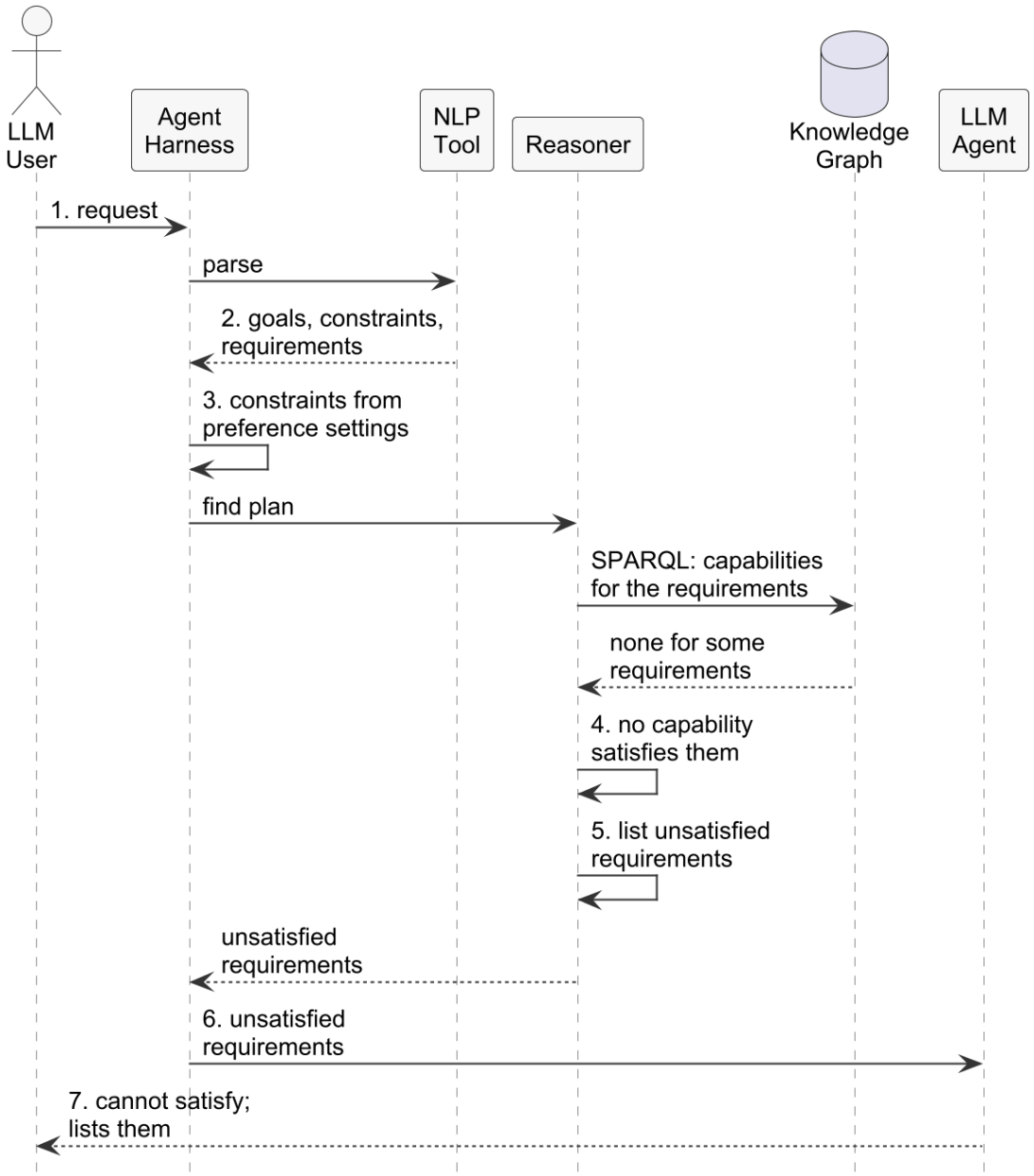
Basic flow Sequence Diagram

Basic Flow of Events (Usage Scenario 1): sequence diagram - AC-001



Alternate Flow Sequence Diagram: Current agentic capabilities cannot satisfy the user's request

Alternate Flow of Events (Usage Scenario 1): sequence diagram - AC-001



VII. Competency Questions

Provide at least 2 competency questions that you will ask of the vocabulary/ontology/knowledge base to implement this use case, including example answers to the questions.

Describe at least one way you expect to use the semantics and/or provenance to propose an answer to the questions. Include an initial description of why the semantics and/or provenance representation and reasoning provides an advantage over other obvious approaches to the problem. (optional – depending on the use case and need for supporting business case).

Competency questions ontology will address are CQs 1, 3, and 5

Purple text means this is where we're leveraging the ontology's DL reasoning and inference to answer the competency question

CQ 1

- **Question:**
  - What steps should the agent follow to answer Ken's request for supplement recommendations, given his preference for few tool calls and low token use?
- **Sample Answer from Ontology:**
  1. Get user profile
  2. Get candidate supplements
     a. Supplement_advisor: recommend_supplement(search_query) => supplement recommendations
  3. Remove redundant supplements
  4. Get supplement products
     a. iHerb-CLI: search(supplement, sort=[price]) => list of supplement products
  5. Rank supplement products by cost
  6. Verify evidence
     a. PubMed: search_articles(search_query) => PMIDs
     b. PubMed: download_articles_batch(PMIDs) => article abstracts
     c. summarize_evidence()
  7. Rank supplement products by evidence
  8. Select supplement product
  9. Produce recommendations
- **Terms used from Ontology:**
  - Request, InformationRequirement, Task, Capability, Skill, Tool, UserConstraints, PreferenceSettings, SoftConstraint, CandidateOrchestrationPlan, PlanEvaluation, DecisionCriterion, ToolCallCount, TokenCost, SelectedOrchestrationPlan, RejectedOrchestrationPlan, OrchestrationWorkflowStep, InformationArtifact.
- **Semantic Processes Involved:**
  - Semantic lookup: identify available Skills and Tools that align with the Capabilities that satisfy the request's InformationRequirements.
  - Reasoning over dependencies: Use the Inputs and Outputs of Skills and Tools to infer dependencies between OrchestrationWorkflowSteps and determine CandidateOrchestrationPlans.
  - Constraint-based reasoning: Identify CandidateOrchestrationPlans that satisfy the request's InformationRequirements and applicable HardConstraints.
  - Multi-criteria plan evaluation: Compare CandidateOrchestrationPlans

using SoftConstraints (ToolCallCount and TokenCost), weighted according to Ken's preferences, and select the highest-ranked valid plan.

- **Usage scenario covered:**
  - Refer to usage scenario 1 (Ken)
- **Description + Ontology Usage**
  - The system retrieves Ken's PreferenceSettings and represents his preferences as SoftConstraints, including minimizing ToolCallCount and TokenCost, alongside the InformationRequirements for recommending supplements, price comparison, evidence verification, and customer reviews.
  - The knowledge graph is queried to identify skills and tools whose capabilities satisfy the InformationRequirements and HardConstraints (if any). The ontology's relationships between Task, Capability, and InformationArtifact are used to infer dependencies between OrchestrationWorkflowSteps, ensuring that required Outputs are available before dependent steps execute.
  - The system then constructs CandidateOrchestrationPlans based on the dependencies of each OrchestrationWorkflowStep
  - The knowledge graph queries the CandidateOrchestrationPlans, dropping those that don't satisfy the InformationRequirements and HardConstraints specified by the user
  - The system evaluates them using the DecisionCriteria (ToolCallCount and TokenCost). The corresponding PlanEvaluation instances record the measured metrics, scores, and rankings. The priorities associated with Ken's soft constraints determine how the criteria contribute to the overall ranking.
  - The reasoner infers that the CandidateOrchestrationPlan whose PlanEvaluation has rank 1 is the SelectedOrchestrationPlan, which the agent executes. Because a Request has at most one SelectedOrchestrationPlan and every CandidateOrchestrationPlan is either selected or rejected, it also infers that Ken's other candidate plan is a RejectedOrchestrationPlan. The knowledge graph records the SelectedOrchestrationPlan, RejectedOrchestrationPlan, and associated evaluation information as provenance, allowing the system to explain why the plan was selected and how it reflects Ken's preferences.

CQ 2

- **Question:**
  - Which workflow steps of a given orchestration should be executed in parallel while satisfying both workflow dependencies and the user's execution preferences?
  - Suppose the generated orchestration plan is:
    1. Get user profile
    2. Get candidate supplements
       a. Supplement_advisor: recommend_supplement() => supplements
    3. Screen interactions

a. OpenFDA: search_drug_adverse_events(medication/supplement)
b. OpenFDA: search_drug_labels(supplement)
c. OpenFDA: search_drug_recalls(supplement)

4. Rank by screening
5. Verify evidence
   a. PubMed: search_articles(supplement/search_query) => PMIDs
   b. PubMed: download_articles_batch(PMIDs) => article abstracts
   c. summarize_evidence()
6. Rank by evidence
7. Get supplement products
   a. iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products
8. Rank supplement products by rating
9. Select supplement product
10. Produce recommendations

- **Sample Answer from Ontology:**
  - Screen interactions
    - OpenFDA: search_drug_adverse_events(medication/supplement)
    - OpenFDA: search_drug_labels(supplement)
    - OpenFDA: search_drug_recalls(supplement)
  - Get supplement products
    - iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products
  - Verify evidence
    - PubMed: search_articles(supplement/search_query) => PMIDs
    - PubMed: download_articles_batch(PMIDs) => article abstracts
    - summarize_evidence()
- **Terms used from Ontology:**
  - SelectedOrchestrationPlan, OrchestrationWorkflowStep, Task, Capability, Skill, Tool, InformationArtifact, CandidateSupplement, EvidenceAssessment, SafetyAssessment, ProductList, consumes, produces, dependsOn.
- **Semantic Processes Involved:**
  - Artifact-based dependency inference: Identify the InformationArtifacts consumed and produced by each Task.
  - Dependency reasoning: Infer that Task B depends on Task A when B consumes an InformationArtifact produced by A.
  - Parallel execution identification: Identify Tasks that share an available Input but have no dependency path between them.
- **Usage scenario covered:**
  - *Refer to usage scenario 1 (Jessica)*
- **Description + Ontology Usage**
  - The ontology traces Jessica's SelectedOrchestrationPlan to its OrchestrationWorkflowStep instances associated with Task instances. Each Task consumes and produces InformationArtifact instances, allowing the system to infer dependencies from the relationship between task Inputs

and Outputs.
    - It examines the Inputs required by the Capabilities Screen interactions, Verify evidence, and Get supplement products tasks. Because all three consume the same CandidateSupplement artifact and none consumes another task's output, the reasoner infers that these tasks have no dependencies on one another.
    - The resulting dependency graph identifies these tasks as eligible for parallel execution. Tasks that require their outputs, such as ranking by screening, ranking by evidence, and selecting a product, must wait until their required artifacts are available.
    - The agent can use this dependency graph to execute independent tasks concurrently while preserving the required ordering of dependent tasks. The ontology records the planned workflow and its dependencies, while the execution trace can record the actual invocations and generated artifacts.

CQ 3

- **Question:**
    - What skills are required to satisfy Jessica's request for supplement recommendations, given she explicitly mentions her diagnosis and prescribed medication?
    - *Query: Provide me with supplement recommendations for building bone mass, while taking into account of my osteoarthritis and prescribed Celebrex medication*
- **Sample Answer from Ontology:**
    - Personalized Supplement Recommendation Skill
    - Supplement Safety Screening Skill (due to her current medication and diagnosed health condition)
    - Evidence Verification Skill
    - Product Selection Skill
- **Terms used from Ontology:**
    - Request, InformationRequirement, Goal, Capability, Skill, UserConstraints, HardConstraint, PreferenceSettings, expresses, wasDerivedFrom, hasCapability, ConstraintSatisfyingSkill, prov:wasAttributedTo
- **Semantic Processes Involved:**
    - Requirement extraction: Parse Jessica's request to identify the Goal and InformationRequirements needed, such as supplement recommendations, scientific evidence, and product selection.
    - Constraint identification: Retrieve applicable UserConstraints, especially her HardConstraints requiring safety screening and evidence verification .
    - Capability matching: Query the knowledge graph to identify capabilities that satisfy each InformationRequirement and HardConstraint
    - Skill mapping: Follow the relationship between Skill and Capability to identify the skills that provide the required capabilities.
- **Usage scenario covered:**

- Refer to usage scenario 1 (Jessica)

- **Description + Ontology Usage**
  - The ontology represents Jessica's request as a Request that expresses a Goal and possibly multiple InformationRequirements (supplement recommendation). Her HardConstraints (including safety screening and evidence verification) are derived from her PreferenceSettings
  - The system queries the knowledge graph to identify Capabilities that satisfy the InformationRequirement and HardConstraints. It then retrieves the Skills associated with those capabilities through the hasCapability relationship.
  - The resulting set includes the Personalized Supplement Recommendation, Supplement Safety Screening, Evidence Verification, and Product Selection skills. Each skill is included because it provides a capability needed to satisfy a requirement or constraint associated with Jessica's request. The reasoner also classifies the Supplement Safety Screening and Evidence Verification skills as ConstraintSatisfyingSkills, because their capabilities satisfy her two HardConstraints. Her Request and her PreferenceSettings are both attributed to her as a prov:Person (prov:wasAttributedTo), which links the request to the settings that her HardConstraints are derived from.

CQ 4

- **Question:**
  - Why were the agent capabilities for a given orchestration invoked in this order?
  - *Query: "Compare magnesium glycinate and magnesium citrate for supporting sleep, and summarize the scientific evidence for each."*
  - *Dante's constraints: minimize execution time, safety screening*
  - Suppose the chosen orchestration plan is:

    1a. Verify evidence
    PubMed: search_articles(magnesium glycinate for sleep) => PMIDs
    PubMed: download_articles_batch(PMIDs) => article abstracts
    summarize_evidence()

    1b. Screen interactions
    OpenFDA: search_drug_adverse_events(magnesium glycinate)
    OpenFDA: search_drug_labels(magnesium glycinate)
    OpenFDA: search_drug_recalls(magnesium glycinate)

    2a. Verify evidence
    PubMed: search_articles(magnesium citrate for sleep) => PMIDs
    PubMed: download_articles_batch(PMIDs) => article abstracts
    summarize_evidence()

    2b. Screen interactions
    OpenFDA: search_drug_adverse_events(magnesium citrate)
    OpenFDA: search_drug_labels(magnesium citrate)
    OpenFDA: search_drug_recalls(magnesium citrate)

    3. Synthesize evidence

- **Sample Answer from Ontology:**
  - One of Dante's constraints is safety screening. Because the Screen interactions skill satisfies that constraint, include it in the orchestration plan. Another one of Dante's constraints is to minimize execution time. Since the Verify evidence and Screen interactions skills require the same dependencies (candidate supplement), they can be invoked in parallel to minimize execution cost. Doing so satisfies both constraints specified by Dante.
- **Terms used from Ontology:**
  - Request, InformationRequirement, Constraint, HardConstraint, SoftConstraint, Capability, Skill, SelectedOrchestrationPlan, OrchestrationWorkflowStep, Task, InformationArtifact, CandidateSupplement, EvidenceAssessment, SafetyAssessment, dependsOn, consumes, produces, DecisionCriterion, Latency, SkillInvocation, ToolInvocation, ExecutionTrace.
- **Semantic Processes Involved:**
  - Dependency reasoning: Infer dependencies between OrchestrationWorkflowSteps from the InformationArtifacts they consume and produce.
  - Parallel-execution identification: Identify OrchestrationWorkflowSteps that do not depend on one another's Outputs and are therefore eligible for concurrent execution.
  - Constraint reasoning: Identify HardConstraint *safety-screening constraint* and the capability required to satisfy it. Identify SoftConstraint *minimize execution time* to justify minimizing execution time
  - Provenance-based explanation: Retrieve the selected plan and its recorded decision information to explain the rationale for the execution order.
- **Usage scenario covered:**
  - Refer to usage scenario 2
- **Description + Ontology Usage**
  - The ontology represents the SelectedOrchestrationPlan as a set of OrchestrationWorkflowStep instances. Each step is associated with the InformationArtifacts it consumes and produces.
  - The ontology uses the consumes and produces relationships to infer dependsOn relationships. Evidence verification produces an EvidenceAssessment, interaction screening produces a SafetyAssessment, and evidence synthesis consumes the assessments required to generate the comparison. This establishes why synthesis must occur after its prerequisite results are available.
  - Dante's safety-screening Constraint is linked to the Capability implemented by the interaction-screening skill, explaining why that skill is included in the plan. His preference to minimize execution time is linked to an execution-time DecisionCriterion, explaining SelectedOrchestrationPlan leverages parallel execution
  - Knowledge graph retrieves the SelectedOrchestrationPlan and its associated DecisionCriterion and ExecutionTrace to explain which steps

were independent, which dependencies constrained their order, and which Constraints and DecisionCriterion motivated the execution strategy.

CQ 5

- **Question:**
  - Why was each agent capability chosen to execute in a given orchestration plan?
  - *Use query, constraints, and orchestration plan from this example from CQ 4*
- **Sample Answer from Ontology:**
  - Verify evidence skill and PubMed: satisfied InformationRequirement "summarize scientific evidence"
  - Screen interactions skill and OpenFDA: satisfied constraint "safety screening"
  - Synthesize evidence skill: satisfied InformationRequirement "compare magnesium glycinate and magnesium citrate"
- **Terms used from Ontology:**
  - Request, InformationRequirement, PreferenceSettings, Constraint, HardConstraint, SoftConstraint, Capability, Skill, Tool, SelectedOrchestrationPlan, OrchestrationWorkflowStep, Task, SkillInvocation, ToolInvocation, satisfiesRequirement, satisfiesConstraint, requires, JustifiedCapability, RequirementSatisfyingCapability, ConstraintSatisfyingCapability.
- **Semantic Processes Involved:**
  - Requirement and constraint retrieval: Retrieve the InformationRequirements and Constraints associated with Dante's Request and PreferenceSettings.
  - Capability tracing: Identify the Capabilities required by each Task in the SelectedOrchestrationPlan
  - Semantic mapping: Follow the satisfiesRequirement and satisfiesConstraint relationships to identify the information requirement or constraint addressed by each capability, respectively.
  - Skill and tool attribution: Identify the Skills and Tools that provides the capability
  - Plan justification: Explain why each OrchestrationWorkflowStep is included by linking its required Capability to a InformationRequirement or Constraint.
- **Usage scenario covered:**
  - Refer to usage scenario 2
- **Description + Ontology Usage**
  - The ontology represents Dante's Request through its associated InformationRequirements and Constraints. The system first queries Dante's SelectedOrchestrationPlan. Then, it queries each OrchestrationWorkflowStep associated with each Task, each of which requires a Capability.
  - For each Task, the knowledge graph retrieves the required Capability and

follows the satisfiesRequirement and satisfiesConstraint relationships to identify the InformationRequirements and Constraints that justify its inclusion, respectively. From the SelectedOrchestrationPlan, we can use the ontology to trace the OrchestrationWorkflowSteps => Task => Capability to identify the Skills and Tools corresponding to the required Capabilities needed to execute

- The explanation will include each OrchestrationWorkflowStep to the corresponding InformationRequirement or Constraint. This allows the system to explain not only which Capabilities were selected, but why each was relevant to Dante's Request. The reasoner classifies each of these Capabilities as a JustifiedCapability: a RequirementSatisfyingCapability when it satisfies one of Dante's InformationRequirements, and a ConstraintSatisfyingCapability when it satisfies his safety-screening HardConstraint.

CQ 6

- **Question:**
  - Why was orchestration plan B selected over orchestration plan A?
  - *Plan A:*

    Get candidate supplements skill

    a. Supplement_advisor: recommend_supplement(heart health) => supplement recommendations

    Verify evidence skill

    b. PubMed: search_articles(search_query) => PMIDs

    c. PubMed: download_articles_batch(PMIDs) => article abstracts

    d. summarize_evidence()

    Screen interactions skill

    e. OpenFDA: search_drug_adverse_events(medication/supplement)

    f. OpenFDA: search_drug_labels(supplement)

    g. OpenFDA: search_drug_recalls(supplement)

    Verify evidence (second, duplicate pass) skill

    h. PubMed: search_articles(search_query) => PMIDs

    i. PubMed: download_articles_batch(PMIDs) => article abstracts

    j. summarize_evidence()

    Rank by evidence

    Select supplement product

    Produce recommendations

    Plan B:

    Get candidate supplements skill

    k. Supplement_advisor: recommend_supplement(heart health) => supplement recommendations

    Verify evidence skill

    l. PubMed: search_articles(search_query) => PMIDs

    m. PubMed: download_articles_batch(PMIDs) => article abstracts

    n. summarize_evidence()

Rank by evidence
Select supplement product
Produce recommendations

- **Sample Answer from Ontology:**
  - Both plans call the Supplement Advisor MCP and verify the candidate supplements against PubMed, so both satisfy Olivia's "strong scientific evidence" constraint. Plan A additionally screens interactions with OpenFDA, which Olivia did not ask for, and runs a second Verify evidence pass with a broader search query, which finds supporting studies for every candidate supplement (evidence coverage 1.00, against 0.70 for Plan B). These extra steps cost five more external tool calls: Plan A makes 8 where Plan B makes 3. Plan B satisfies the evidence constraint with the fewest tool calls, ranks first on the combined DecisionCriteria, and is selected. Plan A is kept in the KG as a rejected candidate together with its scores (see CQ 7).
- **Terms used from Ontology:**
  - Request, InformationRequirement, UserConstraint, SoftConstraint, Capability, Skill, Tool, OrchestrationPlan, CandidateOrchestrationPlan, SelectedOrchestrationPlan, RejectedOrchestrationPlan, PlanEvaluation, DecisionCriterion, PlanMetric, EvidenceCoverage, ToolCallCount, satisfiesConstraint, satisfiesRequirement, prov:wasDerivedFrom, hasScore, hasPriority
- **Semantic Processes Involved:**
  - Candidate plan identification: Retrieve CandidateOrchestrationPlans whose capabilities satisfy the InformationRequirements of the Request.
  - Decision criterion selection: Identify *Evidence Coverage* and *Low Tool Usage* as the DecisionCriteria with the UserConstraints.
  - Plan metric calculation: Calculate each candidate plan's evidence coverage and normalized low-tool-usage scores.
  - Weighted plan evaluation: Combine the DecisionCriterion scores from the PlanMetric using the user's specified priorities; apply equal weights when the priorities are equal.
  - Plan ranking and selection: Rank CandidateOrchestrationPlans by their combined scores and select the highest-ranked eligible CandidateOrchestrationPlan.
  - Provenance recording: Record the SelectedOrchestrationPlan, DecisionCriteria scores, and RejectedOrchestrationPlan so the choice can be explained later.
- **Usage scenario covered:**
  - Refer to usage scenario 3
- **Description + Ontology Usage**
  - The knowledge graph will contain the user's Request, InformationRequirements, and UserConstraints. The reasoner retrieves CandidateOrchestrationPlans whose capabilities satisfy those InformationRequirements and identifies the DecisionCriteria associated with the UserConstraints.

- The system calculates the *Evidence Coverage* and *Low Tool Usage* scores for each candidate plan. *Evidence Coverage* is the share of candidate supplements for which the plan's evidence verification finds supporting studies: 1.00 for Plan A, whose second and broader PubMed pass finds support for every candidate, and 0.70 for Plan B. *Low Tool Usage* is normalized using the number of external tool calls across the CandidateOrchestrationPlans.
- The evaluation process combines the scores according to the user's stated priorities. Since they assign equal priority to both criteria, the scores are added with equal weights. Plan B receives a combined score of 1.70, compared with Plan A's 1.00, and is selected.
- The system records the PlanEvaluation and associated DecisionCriterion in the knowledge graph. This enables the agent to explain why Plan B was preferred, including the trade-off between *evidence coverage* and *external tool usage*.

CQ 7

- **Question:**
  - Which constraints were violated by a candidate orchestration plan?
  - *Refer to candidate plan A from this example* in CQ 6
- **Sample Answer from Ontology:**
  - The candidate plan violated the "minimize external tool calls" constraint. While it did satisfy the "evidence coverage" constraint, it uses some unnecessary tool calls (the three OpenFDA calls screen interactions, which Olivia did not ask for, and the second PubMed pass adds evidence at the cost of two more calls) making it rank lower than other candidate plans.
- **Terms used from Ontology:**
  - Request, UserConstraints, SoftConstraint, OrchestrationPlan, CandidateOrchestrationPlan, SelectedOrchestrationPlan, RejectedOrchestrationPlan, PlanEvaluation, DecisionCriterion, PlanMetric, ToolCallCount, EvidenceCoverage, prov:wasDerivedFrom, hasPriority
- **Semantic Processes Involved:**
  - Candidate plan retrieval: Retrieve RejectedOrchestrationPlan A and its PlanEvaluation alongside the SelectedOrchestrationPlan B.
  - Metric comparison: Compare the OrchestrationPlans' scores for each DecisionCriterion.
  - Constraint tracing: Trace the *Low Tool Usage* DecisionCriterion to the SoftConstraint to *minimize external tool calls*.
  - Preference-aware interpretation: Interpret the score difference of Olivia's equal priorities for *evidence coverage* and *low tool usage*.
  - Explanation generation: Identify the *minimize external tool calls* SoftConstraint was not satisfied by OrchestrationPlan A and provide the corresponding scores and comparison of both OrchestrationPlan A and B with their PlanEvaluations (*Low Tool Usage* as the DecisionCriterion).
- **Usage scenario covered:**
  - Refer to usage scenario 3

- **Description + Ontology Usage**
  - The knowledge graph stores CandidateOrchestrationPlans and their PlanEvaluation results for each DecisionCriterion.
  - The system retrieves RejectedOrchestrationPlan A's PlanEvaluation and compares its scores with SelectedOrchestrationPlan B's PlanEvaluation. It traces the relationship between the DecisionCriterion and the SoftConstraint it measures to identify the preference that Plan A inadequately satisfies.
  - Plan A's *low-tool-usage* score of 0.00, compared with Plan B's 1.00, indicates that Plan A performs worse against Olivia's SoftConstraint to *minimize external tool call*s. Its EvidenceCoverage score is higher, but Olivia's equal priorities expressed by hasPriority indicates the two DecisionCriteria must be considered together.
  - The system returns the identified SoftConstraint, the supporting evaluation scores, and the comparison with the selected plan. The explanation can be traced by the CandidateOrchestrationPlans' PlanEvaluation using *Low Tool Usage* as the DecisionCriterion

CQ 8

- **Question:**
  - What skills depend on other skills?
- **Sample Answer from Ontology:**
  - The Evidence Verification Skill and the Supplement Safety Screening Skill depend on the Personalized Supplement Recommendation Skill, because the tasks they perform consume the CandidateSupplement that its task produces. The Product Selection Skill depends on three skills: the Personalized Supplement Recommendation Skill (CandidateSupplement), the Evidence Verification Skill (EvidenceAssessment) and the Supplement Safety Screening Skill (SafetyAssessment). The Personalized Supplement Recommendation Skill depends on no other skill.
- **Terms used from Ontology:**
  - Skill, InformationArtifact, consumes, produces, dependsOn
- **Semantic Processes Involved:**
  - Skill discovery: Retrieving available Skills from the knowledge graph.
  - Artifact retrieval: Identify the InformationArtifacts each Skill consumes and produces.
  - Dependency inference: Infer that Skill B dependsOn Skill A when Skill A produces an InformationArtifact required by Skill B.
- **Usage scenario covered:**
  - No specific usage scenario. Used for testing ontology's general correctness
- **Description + Ontology Usage**
  - The ontology represents Skills and the InformationArtifacts they consume and produce. The system queries these relationships using the property chain dependsOn to infer dependencies.
  - For example, CandidateSupplement is produced by the Personalized Supplement Recommendation Skill and consumed by both the Evidence

Verification Skill and Supplement Safety Screening Skill. Their outputs, EvidenceAssessment and SafetyAssessment, are then consumed by the Product Selection Skill.
    - The knowledge graph can use these relationships to construct a directed skill-dependency graph and answer dependency queries. The inferred graph supports execution ordering. Skills that share an input but do not depend on each other's outputs may execute in parallel.

CQ 9

- **Question:**
    - Are all the prerequisites satisfied before executing the Evidence Verification Skill given the agent's current phase in Jessica's orchestration?
    - *Example: about to execute Evidence Verification Skill when:*
        - *CandidateSupplement ✗*
        - *SearchQuery ✓*
- **Sample Answer from Ontology:**
    - No, the agent cannot execute the Evidence Verification Skill because not all the prerequisites have been satisfied (namely CandidateSupplement). The agent must call the Personalized Supplement Recommendation Skill, which returns a CandidateSupplement before executing the Evidence Verification Skill.
- **Terms used from Ontology:**
    - Skill, Precondition, hasPrecondition, agentO:Task, performedBySkill, consumes, RequiredInput, SelectedOrchestrationPlan, OrchestrationWorkflowStep, InformationArtifact, Invocation, SkillInvocation, ExecutionTrace, prov:wasGeneratedBy, prov:wasAssociatedWith, pplan:correspondsToStep
- **Semantic Processes Involved:**
    - Identify the pending skill execution: Retrieve the Skill and the agentO:Task associated with the SelectedOrchestrationPlan's WorkflowStep.
    - Check skill preconditions: Retrieve the skill's Precondition instances through hasPrecondition and determine whether they hold.
    - Identify required inputs: Retrieve the InformationArtifacts consumed by the agentO:Task and identify which are classified as RequiredInput.
    - Inspect execution provenance: Query the ExecutionTrace and the prov:wasGeneratedBy relationships to determine which required artifacts have been generated.
    - Determine readiness: Mark the Skill as ready only if its Preconditions hold and all required input artifacts have generation records.
- **Usage scenario covered:**
    - Refer to usage scenario 1 (Jessica)
- **Description + Ontology Usage**
    - A Skill has Preconditions (hasPrecondition). Each OrchestrationWorkflowStep's agentO:Task is performed by a Skill

(performedBySkill) and consumes and produces InformationArtifacts. A RequiredInput is an Input that fills a Parameter with isRequired true. It is distinct from a Precondition.

- The agent's current phase is the ExecutionTrace, which records the Invocations run so far. An Invocation can confirm a Precondition (confirmsPrecondition), and an artifact links to the Invocation that generated it (prov:wasGeneratedBy).
- The query finds the step in the SelectedOrchestrationPlan whose task is performed by the Skill. It then checks that each Precondition is confirmed, and each RequiredInput was generated, by an Invocation the trace records. Any missing input is traced back to the step and Skill that produce it.

VIII. Resources

In order to support the capabilities described in this Use Case, a set of resources must be available and/or configured. These resources include the set of actors listed above, with additional detail, and any other ancillary systems, sensors, or services that are relevant to the problem/use case.

Knowledge Bases, Repositories, or other Data Sources

Data	Type	Characteristics	Description	Owner	Source	Access Policies & Usage
(dataset or repository name)	*(remote, local/in situ, etc.)*	*e.g. – no cloud cover*	*Short description of the dataset, possibly including rationale of the usage characteristics*		*Source (possibly a system, or remote site) for discovery and access*	
PubMed MCP Server	*remote*	*Third-party MCP server*	*Connects to research repository for retrieving biomedical -related papers*	*aeghnnsw*	*https://github.com/aeghnnsw/pubmed-mcp*	*Open-source Github License - must comply with PubMed's terms of service for API usage guidelines*
Supplement Advisor MCP Server	*remote*	*Third-party MCP server*	*Recommends supplements based on user's needs*	*erinheit451*	*https://github.com/erinheit451/supplement-advisor-mcp*	*Open-source Github MIT license*
OpenFDA MCP Server	*remote*	*Third-party MCP server*	*Provides safety and regulatory information on FDA-regulated*	*Augmented Nature*	*https://github.com/Augmented-Nature/OpenFDA-MCP-Server*	*Open-source Github license*

			products (recalls, adverse effects, shortages)			
iHerb-CLI	*remote*	*Third-party Client*	*Provides pricing and reviews for supplements from iHerb*	*SeverinAlexB*	*https://github.com/SeverinAlexB/iherb-cli*	*Open-source github MIT license*

External Ontologies, Vocabularies, or other Model Services

Resource	Language	Description	Owner	Source	Describes/Uses	Access Policies & Usage
(ontology, vocabulary, or model name)	*(ontology language and syntactic form, e.g., RDFS - N3)*	*If the service is one that runs a given ontology or model-based application at a given frequency, state that in addition to the basic description*		*Source (link to the registry or directly to the ontology, vocabulary, or model where that model is maintained, if available)*	*List of one or more data sources described by and/or used by the model*	
Representing Agentic Tools in Knowledge Graphs for Structure-Aware Tool Discovery Under Tool Overload	Paper that proposes high-level ontology (core classes, relationships, taxonomies) No access to github or owl/rdf file	Proposes ontology and KG as layer for tool discovery	Isaiah Onando Mulang' ~Isaiah_Onando_Mulang'1 , Johannes Thaller, Tushar Trivedi, Lars Heling, Felix Sasaki	https://openreview.net/forum?id=7MVoH9Y3mi&noteId=LGfMuSRiu6	Use cases: Discovering tools	Creative Commons License Attribution 4.0 International (CC BY 4.0).
AgentO	Ontology file (RDF/OWL)	Formalizes core concepts, roles, and relationships necessary to describe contemporary agent-based and workflow-	Kabul Kurniawan, Fajar J. Ekaputra	https://sepses.ifs.tuwien.ac.at/onto/index-en.html	Use cases: Workflow reconstruction Workflow reuse Workflow auditing Captured classes: Agent, Task,	Creative Commons License Attribution 4.0 International (CC BY 4.0).

		oriented AI systems, with particular emphasis on LLM-driven agent frameworks			Tool, Goal, Role, Capability, Workflow Pattern, Workflow Step	

Other Resources, Service, or Triggers *(e.g., event notification services, application services, etc.)*

Resource	Type	Description	Owner	Source	Access Policies & Usage
(sensor or external service name)		*Include a description of the resource as well as availability, if applicable*	*Primary owner of the service*	***Application or service URL**; if subscription based, include subscription and any subscription owner*	
Open WebUI	*Agent Harness*	*Web interface to connect AI models* *Supports MCP and Agent Skills*	*Tim J. Baek and team*	*https://knowledge.exlibrisgroup.com/Ulrich's/Product_Documentation/Configuring/Ulrichsweb_API/Ulrichsweb%3A_Using_the_Ulrichsweb_API*	*Open WebUI License*

IX. References and Bibliography

List all reference documents – policy documents, regulations, standards, de-facto standards, glossaries, dictionaries and thesauri, taxonomies, and any other reference materials considered relevant to the use case

[1] I. O. Mulang', J. Thaller, T. Trivedi, L. Heling, and F. Sasaki, "Representing agentic tools in knowledge graphs for structure-aware tool discovery under tool overload," in Proc. 2nd Workshop on Knowledge Graphs and Agentic Systems Interplay (NORA 2026), co-located with IJCAI-ECAI 2026, Bremen, Germany, 2026. [Online]. Available: https://openreview.net/forum?id=7MVoH9Y3mi

[2] A. Ekelhart, K. Kurniawan, F. J. Ekaputra, and E. Kiesling, "AgentO: An ontology for modeling agentic AI systems," in The Semantic Web: 23rd European Semantic Web Conference, ESWC 2026, Proceedings, Part II, Lecture Notes in Computer Science, vol. 16550, M. Acosta et al., Eds. Cham, Switzerland: Springer, 2026, pp. 298–320, doi: 10.1007/978-3-032-25159-6_16.

[3] K. Kurniawan and F. J. Ekaputra, "AgentO: An ontology for modeling agentic AI systems," ontology release v0.2, Zenodo, Jan. 22, 2026, doi: 10.5281/zenodo.18342625. [Online]. Available: https://w3id.org/agentic-ai/onto

[4] Model Context Protocol Project, "Model Context Protocol specification," revision 2026-07-28, LF

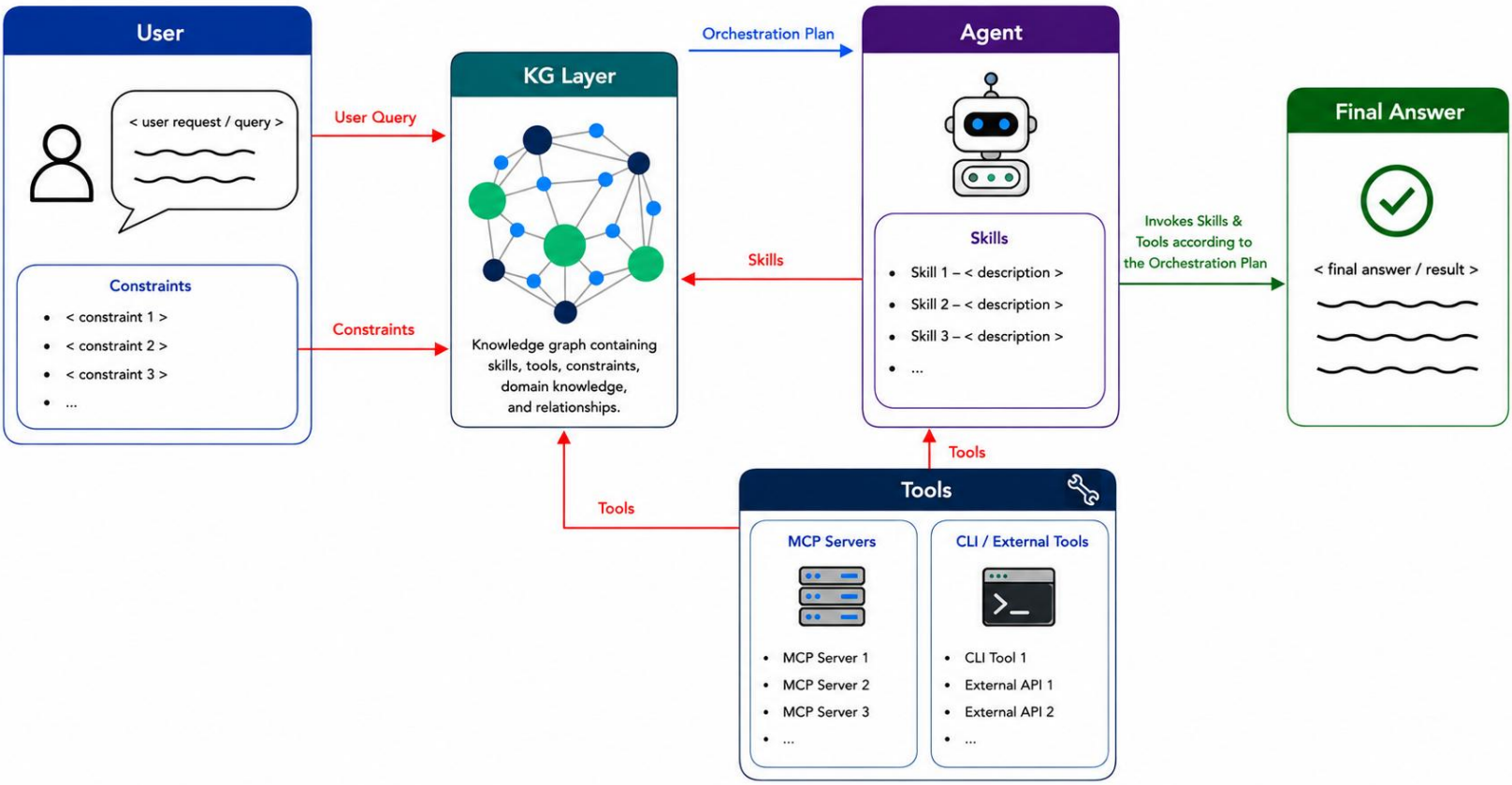
Projects, LLC, Jul. 28, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28
[5] X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, et al., “SkillsBench: Benchmarking how well agent skills work across diverse tasks,” arXiv:2602.12670, Feb. 2026. [Online]. Available: https://arxiv.org/abs/2602.12670
[6] H. Xu, C. Li, X. Ma, X. Ou, Z. Zhang, T. He, et al., “The evolution of tool use in LLM agents: From single-tool call to multi-tool orchestration,” arXiv:2603.22862, Mar. 2026. [Online]. Available: https://arxiv.org/abs/2603.22862
[7] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, “Understanding the planning of LLM agents: A survey,” arXiv:2402.02716, Feb. 2024. [Online]. Available: https://arxiv.org/abs/2402.02716
[8] D. Martin et al., “OWL-S: Semantic markup for web services,” W3C Member Submission, Nov. 22, 2004. [Online]. Available: https://www.w3.org/submissions/OWL-S/
[9] T. Lebo, S. Sahoo, and D. McGuinness, Eds., “PROV-O: The PROV ontology,” W3C Recommendation, Apr. 30, 2013. [Online]. Available: https://www.w3.org/TR/prov-o/
[10] National Institute of Standards and Technology, "Lessons learned from the consortium: Tool use in agent systems," Aug. 5, 2025. [Online]. Available: https://www.nist.gov/news-events/news/2025/08/lessons-learned-consortium-tool-use-agent-systems
[11] IBM Research, "AI Atlas Nexus: AI risk ontology." [Online]. Available: https://ibm.github.io/ai-atlas-nexus/ontology/
[12] D. Garijo and Y. Gil, "The P-Plan ontology," version 1.3, Mar. 12, 2014. [Online]. Available: http://purl.org/net/p-plan
[13] N. Noy and A. Rector, Eds., "Defining N-ary relations on the Semantic Web," W3C Working Group Note, Apr. 12, 2006. [Online]. Available: https://www.w3.org/TR/swbp-n-aryRelations/
[14] J. Bao, E. F. Kendall, D. L. McGuinness, and P. F. Patel-Schneider, Eds., "OWL 2 Web Ontology Language quick reference guide (second edition)," W3C Recommendation, Dec. 11, 2012. [Online]. Available: https://www.w3.org/TR/owl2-quick-reference/

X. Notes

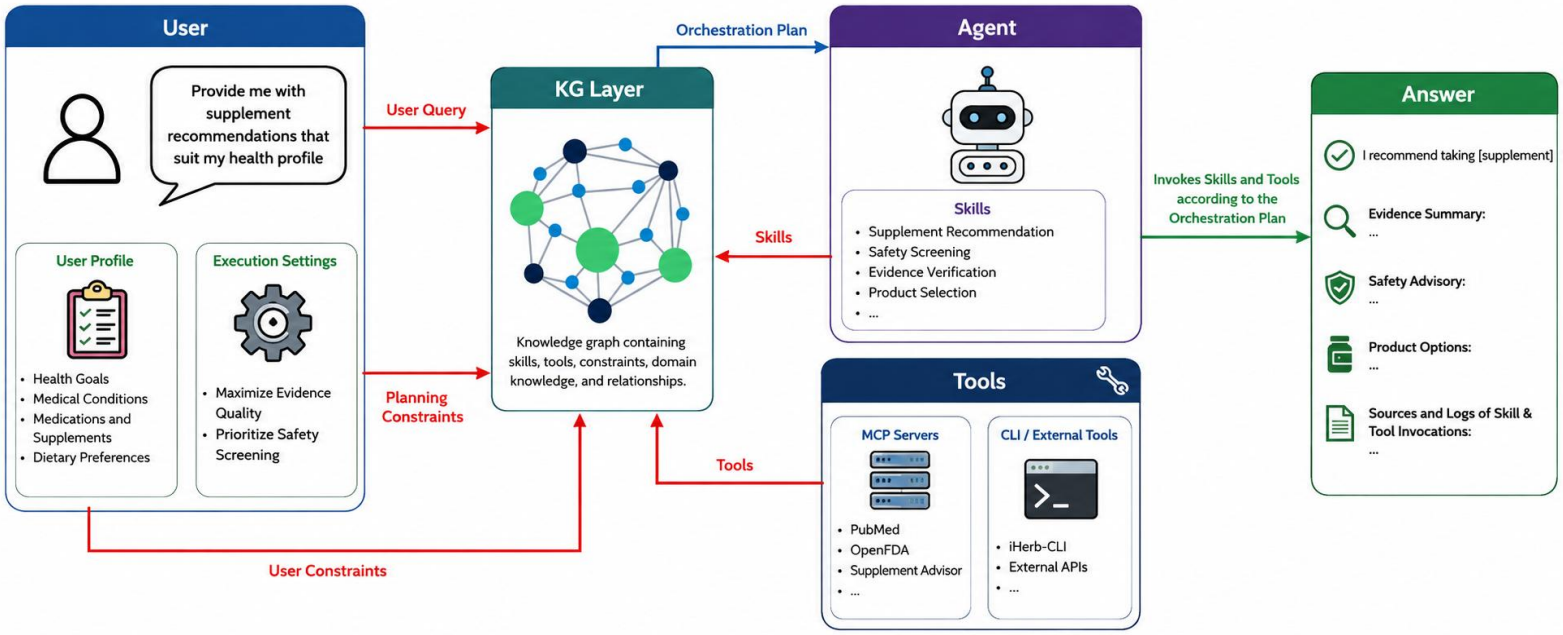
There is always some piece of information that is required that has no other place to go. This is the place for that information.

Concept of Operations

For a general ontology-guided agentic orchestration use case



For a specific scope: personalized supplement recommendation



Future work

- Ontology will consider user-aware personalized constraints

- Development of NLP parser
- Incorporate Explanation Ontology for why an orchestration plan was selected over other candidate plans